

2.14 Trees

2.14.1 Diametro

El diámetro de un árbol se puede hallar haciendo un BFS desde cualquier nodo, se toma el nodo u mas lejano que haya sido encontrado y se hace otro BFS usando u como raíz, entonces se tomo el nodo v mas lejano. Los nodos u y v son el diámetro. Adicionalmente, para hallar la distancia mas larga de cualquier nodo x a otro, esta distancia estará dada por $\max(dv[x], du[x])$.

2.14.2 Subtrees and paths

```
// - Si nos dan un arbol rooteado y tenemos que hacer queries de
algun tipo sobre
// algunos sub arboles (como sumar los valores de cada
elemento de algun subarbol)
// y tenemos que hacer updates sobre los valores de los nodos,
se puede construir
// un array de modo que todos los elementos de cualquier
subarbol sean continuos
// y se puede guardar cual es el rango de cada subarbol
mediante un DFS. Una vez
// se ha procesado todo esto y generado el nuevo array, se
pueden realizar queries
// y updates de cualquier tipo usando un segment tree.
// - Los rangos se guardaran  $ran[u] = [l, r)$ 
// - Si no necesitamos hallar valores entre el subarbol de algun
nodo sino en su
// camino hacia la raiz entonces hacemos updates en rango y
querry individuales,
// es decir, el valor de un nodo se le suma a todo su
correspondiente subarbol, y
// para hacer una query entre el camino de un nodo hacia la raiz,
se hace la query
```

```
// especificamente sobre el nodo que se pregunta (query
individual).
```

```
const int maxn = 2e5 + 1;
vector<vector<int>>> tree(maxn);
```

```
void DFS(vector<pii>& ran, int u, int pa, int &cnt){
    // Se dice que el rango de u inicia en cnt. Ademas ese sera su
indice en el array que se contruira.
    ran[u].fi = cnt;
    cnt++; // Se suma 1 para guardar que se ha visitado un nuevo
nodo
    for(int v : tree[u]){
        if(v == pa) continue;
        DFS(ran, v, u, cnt); // Se visita el subarbol que tiene u como
raiz
    }

    // Despues de visitar los nodos que pertenecen al subarbol de
u, podemos decir en que indice termina el rango de u
    ran[u].se = cnt;
}
```

2.14.3 LCA con DFS y RMQ

/*

LCA usando Euler Tour Technique.

Consiste en utilizar un DFS para marcar la primera vez que se visita cada nodo (start), su profundidad (depth) y tener un array (id) en el que se inserta un valor u cada que dicho nodo es visitado en el DFS, es decir, cuando se visita por primera vez y cuando se termina de visitar un hijo, lo cual nos dice que

cada nodo sera agregado 1 vez por cada hijo que tiene mas una por si mismo, por tanto el array id tendra $2*n - 1$ elementos.

Para hacer las queries de LCA(u, v) se necesita hallar el nodo con menor profundidad en el intervalo

[start[u], start[v]], con $start[u] \leq start[v]$, lo cual se hara con una sparse table de minimos. La

sparse table se rellenara con los pares {depth[id[i]], id[i]}, para $0 \leq i < sz(id)$. Cada query halla el nodo con menor profundidad en ese intervalo en $O(1)$.

Para hallar la distancia entre 2 nodos del arbol se puede usar la siguiente formula:

$distancia(u, v) = depth[u] + depth[v] - 2*depth[LCA(u, v)]$
*/

```
const int maxn = 2e5 + 1;
vector<vector<int>> tree(maxn);
vector<int> id, depth(maxn), start(maxn);
vector<vector<pii>> rmq;
```

// Llenar sparse table

// $O(n \lg(n))$

```
void fillRMQ(){
```

```
    int n = sz(id); rmq.resize(n);
```

```
    int lgn = 31 - __builtin_clz(n);
```

```
    for(int i = 0; i < n; i++){
```

```
        rmq[i].resize(lgn + 1);
```

```
        rmq[i][0] = {depth[id[i]], id[i]};
```

```
    }
```

```
    for(int j = 1; j <= lgn; j++){
        for(int i = 0; i + (1 << j) - 1 < n; i++){
            rmq[i][j] = min(rmq[i][j - 1], rmq[i + (1 << (j - 1))][j - 1]);
        }
    }
```

// Halla el valor minimo en el intervalo [l, r]

// $O(1)$

```
pii query(int l, int r){
```

```
    int lg = 31 - __builtin_clz(r - l + 1); // Piso del Log2 del tamaño del rango
```

```
    return min(rmq[l][lg], rmq[r - (1 << lg) + 1][lg]); // [l, r]
```

```
}
```

// Halla los valores correspondientes de los arrays depth, id y start

```
void DFS(int u, int pa){
```

```
    start[u] = sz(id); // Marcar en que momento se visito por primera vez
```

```
    id.pb(u); // Insertar cuando se visita el nodo pro primera vez
```

```
    for(int v : tree[u]){
```

```
        if(v == pa) continue;
```

```
        depth[v] = depth[u] + 1;
```

```
        DFS(v, u);
```

```
        id.pb(u); // Insertar cada vez que se regresa al nodo
```

```
    }
```

```
}
```

// Halla el LCA entre 2 nodos

// $O(1)$

```
int LCA(int a, int b){
```

```

    if(start[a] > start[b]) swap(a, b); // Se asegura que start[a]
corresponda a l y start[b] a r
    return query(start[a], start[b]).se;
}

```

```

void solver(){
    int n; cin>>n;
    for(int i = 1; i < n; i++){
        int u, v; cin>>u>>v; u--; v--;
        tree[u].pb(v);
        tree[v].pb(u);
    }
}

```

```

DFS(0, -1);
fillRMQ();

```

```

for(int i = 0; i < n-1; i++){
    for(int j = i+1; j < n; j++)
        cout<<"El Lowest Common Ancestor entre "<<i<<" y
"<<j<<" es: "<<LCA(i, j)<<endl;
    }
}

```


4.4 Lazy propagation nueva

/*

Segment Tree Lazy Propagation (ejemplo con update assign y calc suma.)

- Es necesario que se cumpla la propiedad asociativa tanto en las operaciones de update como calc.

- Es necesario que se cumpla que $\text{calcOp}(\text{updateOp}(a, x), \text{updateOp}(b, x)) == \text{updateOp}(\text{calcOp}(a, b), x)$,

es decir, que updateOp sea distributiva relativa a calcOp , ejemplo, update de multiplicacion, calc

suma. En caso de que no se cumpla esa propiedad (update assign, calc suma o update suma, calc suma),

se debe ajustar utilizando la longitud del rango en la operacion update.

- Si se llega a pedir update de diferentes tipos, toca tener cuidado con la propagacion. Ejemplo con

update de asignacion (1) y de suma (0):

Updates:

```
ll updateOp(ll a, ll b, ll len, int op){ // op -> assign, !op -> suma
    if(op == -1) return neutro;
    if(b == neutro) return a;
    if(a == neutro) return b * len;
    return op ? b * len : a + b * len;
}
```

Propagacion:

```
void propagate(int v, int tl, int tr){
    if(tr - tl == 1 || lazy[v].se == -1) return;
    int tm = (tr + tl) / 2;
```

```
    lazy[2*v + 1].fi = updateOp(lazy[2*v + 1].fi, lazy[v].fi, 1,
    lazy[v].se);
    if(lazy[2*v + 1].se == -1) lazy[2*v + 1].se = lazy[v].se;
```

```
    else lazy[2*v + 1].se = min(1, lazy[2*v + 1].se + lazy[v].se);
```

```
    tree[2*v + 1] = updateOp(tree[2*v + 1], lazy[v].fi, tm - tl,
    lazy[v].se);
```

```
    lazy[2*v + 2].fi = updateOp(lazy[2*v + 2].fi, lazy[v].fi, 1,
    lazy[v].se);
    if(lazy[2*v + 2].se == -1) lazy[2*v + 2].se = lazy[v].se;
    else lazy[2*v + 2].se = min(1, lazy[2*v + 2].se + lazy[v].se);
```

```
    tree[2*v + 2] = updateOp(tree[2*v + 2], lazy[v].fi, tm - tl,
    lazy[v].se);
```

```
    lazy[v] = {neutro, -1};
```

```
}
```

*/

```
class segTree {
private:
```

```
    int size;
    vector<ll> lazy;
    vector<ll> tree;
```

```
    ll neutro = LLONG_MAX - 1;
```

```
    ll updateOp(ll a, ll b, ll len){
        if(b == neutro) return a;
        return b * len;
    }
```

```
    ll calcOp(ll a, ll b){
        if(a == neutro) return b;
        if(b == neutro) return a;
```

```

    return a + b;
}

// Para query suma y update suma
// ll updateOp(ll a, ll b, ll len){
//   if(b == neutro) return a;
//   if(a == neutro) return b*len;
//   return a + b*len;
// }

// ll calcOp(ll a, ll b){
//   if(a == neutro) return b;
//   if(b == neutro) return a;
//   return a + b;
// }

// Para query minimo y update suma
// ll updateOp(ll a, ll b, ll len){
//   if(b == neutro) return a;
//   if(a == neutro) return b;
//   return a + b;
// }

// ll calcOp(ll a, ll b){
//   if(a == neutro) return b;
//   if(b == neutro) return a;
//   return min(a, b);
// }

void applyUpdOp(ll &a, ll b, ll len){
    a = updateOp(a, b, len);

```

```

}

// O(1)
void propagate(int v, int tl, int tr){
    if(tr - tl == 1) return;
    int tm = (tr + tl) / 2;
    // Para Update de invertir (cambiar 0s por 1s y viceversa),
    usar:
    // lazy[2*v + 1] = !lazy[2*v + 1];
    // lazy[2*v + 2] = !lazy[2*v + 2];
    applyUpdOp(lazy[2*v + 1], lazy[v], 1);
    applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
    applyUpdOp(lazy[2*v + 2], lazy[v], 1);
    applyUpdOp(tree[2*v + 2], lazy[v], tr - tm);
    lazy[v] = neutro;
}

// O(lg(n))
// [l, r)
void update(int l, int r, ll val, int v, int tl, int tr){
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return;
    if(tl >= l && tr <= r){
        applyUpdOp(lazy[v], val, 1);
        applyUpdOp(tree[v], val, tr - tl);
        return;
    }

    int tm = (tl + tr) / 2;
    update(l, r, val, 2*v + 1, tl, tm);
    update(l, r, val, 2*v + 2, tm, tr);
    tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);

```

```

}

// O(lg(n))
// [l, r)
ll calc(int l, int r, int v, int tl, int tr){
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return neutro;
    if(tl >= l && tr <= r) return tree[v];

    int tm = (tl + tr) / 2;
    ll m1 = calc(l, r, 2*v + 1, tl, tm);
    ll m2 = calc(l, r, 2*v + 2, tm, tr);
    return calcOp(m1, m2);
}

// O(n)
void build(vector<ll>& a, int v, int tl, int tr){
    if(tr == tl + 1){
        tree[v] = a[tl];
        return;
    }
    int tm = (tr + tl) / 2;
    build(a, 2*v + 1, tl, tm);
    build(a, 2*v + 2, tm, tr);
    tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
}

public:
void init(int n){
    size = 1;
    while(size < n) size *= 2;
    lazy.assign(2*size, neutro);

```

```

    tree.assign(2*size, 0LL);
}

void update(int l, int r, ll val){
    update(l, r, val, 0, 0, size);
}

ll calc(int l, int r){
    return calc(l, r, 0, 0, size);
}

void build(vector<ll>& a){
    build(a, 0, 0, size);
}
};

```

4.5 Lazy Kth one

// Segment Tree Lazy Propagation

// - Halla el Kth one en 0-index, es decir, si me piden el indice del 0th one, me estan pididiendo el indice

// del primer one.

// - El array tambien es 0-index, asi que si estuviera lleno de ones, se considera que el

// 0th one esta en la posicion 0.

// - Las updates son de invertir (los 0s pasan a ser 1s y viceversa).

```

int size;
vector<bool> lazy;
vector<int> tree;

int updateOp(int a, int len){
    return len - a;
}

```

```

}

int calcOp(int a, int b){
    return a + b;
}

void applyUpdOp(int &a, int len){
    a = updateOp(a, len);
}

// O(1)
void propagate(int v, int tl, int tr){
    if(tr - tl == 1 || !lazy[v]) return;
    int tm = (tr + tl) / 2;
    lazy[2*v + 1] = !lazy[2*v + 1];
    lazy[2*v + 2] = !lazy[2*v + 2];
    applyUpdOp(tree[2*v + 1], tm - tl);
    applyUpdOp(tree[2*v + 2], tr - tm);
    lazy[v] = 0;
}

// O(lg(n))
// [l, r)
void update(int l, int r, int v, int tl, int tr){
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return;
    if(tl >= l && tr <= r){
        lazy[v] = 1;
        applyUpdOp(tree[v], tr - tl);
        return;
    }

```

```

    int tm = (tl + tr) / 2;
    update(l, r, 2*v + 1, tl, tm);
    update(l, r, 2*v + 2, tm, tr);
    tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
}

// O(lg(n))
// [l, r)
int calc(int l, int r, int v, int tl, int tr, int idx){
    propagate(v, tl, tr);
    if(tr == tl + 1) return tl;

    int tm = (tl + tr) / 2;
    if(tree[2*v + 1] >= idx) return calc(l, r, 2*v + 1, tl, tm, idx);
    return calc(l, r, 2*v + 2, tm, tr, idx - tree[2*v + 1]);
}

```

4.6 Lazy primer elemento mayor a X

// Segment Tree Lazy Propagation

// - Se hacen queries l, x, donde se retorna el indice (0-index) del primer elemento que cumpla

// i >= l && a[i] >= x.

// - En esta implemetacion se usan updates de suma.

```

int size;
vector<ll> lazy;
vector<ll> tree;

ll neutro = LLONG_MAX - 1;

ll updateOp(ll a, ll b, ll len){
    if(b == neutro) return a;
    if(a == neutro) return b;

```



```

    return a + b;
}

ll calcOp(ll a, ll b){
    if(a == neutro) return b;
    if(b == neutro) return a;
    return max(a, b);
}

void applyUpdOp(ll &a, ll b, ll len){
    a = updateOp(a, b, len);
}

// O(1)
void propagate(int v, int tl, int tr){
    if(tr - tl == 1) return;
    int tm = (tr + tl) / 2;
    applyUpdOp(lazy[2*v + 1], lazy[v], 1);
    applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
    applyUpdOp(lazy[2*v + 2], lazy[v], 1);
    applyUpdOp(tree[2*v + 2], lazy[v], tr - tm);
    lazy[v] = neutro;
}

// O(lg(n))
// [l, r)
void update(int l, int r, ll val, int v, int tl, int tr){
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return;
    if(tl >= l && tr <= r){
        applyUpdOp(lazy[v], val, 1);
        applyUpdOp(tree[v], val, tr - tl);
    }
}

```

```

    return;
}

int tm = (tl + tr) / 2;
update(l, r, val, 2*v + 1, tl, tm);
update(l, r, val, 2*v + 2, tm, tr);
tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
}

// O(lg(n))
// [l, r)
int calc(int l, int r, int v, int tl, int tr, ll x){
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return -1;
    if(tr == tl + 1){
        return tl;
    }

    int tm = (tl + tr) / 2;
    int ans = -1;
    if(tree[2*v + 1] >= x) ans = calc(l, r, 2*v + 1, tl, tm, x);
    if(ans == -1 && tree[2*v + 2] >= x) ans = calc(l, r, 2*v + 2, tm, tr,
x);
    return ans;
}

```

4.7 Lazy segmento con mayor suma

// Update de asignacion, query de segmento con mayor suma
// La respuesta se guarda en .seg

```

struct Node{
    ll seg, pref, suf, sum;
}

```

```

};

class segTree {
private:
    int size;
    vector<Node> lazy;
    vector<Node> tree;

    Node neutro = {0LL, 0LL, 0LL, LLONG_MAX - 1};

    Node updateOp(Node a, Node b, ll len){
        if(b.sum == neutro.sum) return a;
        Node ans = {0LL, 0LL, 0LL, 0LL};
        ans.sum = b.sum * len;
        if(b.sum > 0LL) ans.seg = ans.suf = ans.pref = ans.sum;

        return ans;
    }

    Node calcOp(Node a, Node b){
        if(a.sum == neutro.sum) return b;
        if(b.sum == neutro.sum) return a;
        Node ans;
        ans.seg = max(0LL, max(max(a.seg, b.seg), a.suf + b.pref));
        ans.pref = max(0LL, max(a.pref, a.sum + b.pref));
        ans.suf = max(0LL, max(b.suf, b.sum + a.suf));
        ans.sum = a.sum + b.sum;

        return ans;
    }

    void applyUpdOp(Node &a, Node b, ll len){

```

```

        a = updateOp(a, b, len);
    }

    // O(1)
    void propagate(int v, int tl, int tr){
        if(tr - tl == 1) return;
        int tm = (tr + tl) / 2;
        applyUpdOp(lazy[2*v + 1], lazy[v], 1);
        applyUpdOp(tree[2*v + 1], lazy[v], tm - tl);
        applyUpdOp(lazy[2*v + 2], lazy[v], 1);
        applyUpdOp(tree[2*v + 2], lazy[v], tr - tm);
        lazy[v] = neutro;
    }

    // O(lg(n))
    // [l, r)
    void update(int l, int r, ll val, int v, int tl, int tr){
        propagate(v, tl, tr);
        if(tl >= r || l >= tr) return;
        if(tl >= l && tr <= r){
            Node x = {0LL, 0LL, 0LL, val};
            applyUpdOp(lazy[v], x, 1);
            applyUpdOp(tree[v], x, tr - tl);
            return;
        }

        int tm = (tl + tr) / 2;
        update(l, r, val, 2*v + 1, tl, tm);
        update(l, r, val, 2*v + 2, tm, tr);
        tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
    }

```

```

// O(lg(n))
// [l, r)
Node calc(int l, int r, int v, int tl, int tr){
    propagate(v, tl, tr);
    if(tl >= r || l >= tr) return neutro;
    if(tl >= l && tr <= r) return tree[v];

    int tm = (tl + tr) / 2;
    Node m1 = calc(l, r, 2*v + 1, tl, tm);
    Node m2 = calc(l, r, 2*v + 2, tm, tr);
    return calcOp(m1, m2);
}

```

```

// void build(int v, int tl, int tr){ // O(n)
//     if(tr == tl + 1){
//         tree[v] = 1;
//         return;
//     }
//     int tm = (tr + tl) / 2;
//     build(2*v + 1, tl, tm);
//     build(2*v + 2, tm, tr);
//     tree[v] = calcOp(tree[2*v + 1], tree[2*v + 2]);
// }

```

public:

```

void init(int n){
    size = 1;
    while(size < n) size *= 2;
    lazy.assign(2*size, {0LL, 0LL, 0LL, 0LL});
    tree.assign(2*size, {0LL, 0LL, 0LL, 0LL});
    // build(0, 0, size);
}

```

```

void update(int l, int r, ll val){
    update(l, r, val, 0, 0, size);
}

Node calc(int l, int r){
    return calc(l, r, 0, 0, size);
}
};

```